

Wasla Backend - Full Technical Documentation

1. Project Overview

Wasla Backend is a robust server-side application designed to power a skill and service exchange platform. It follows a modular architecture using Node.js, Express, and TypeScript. The application relies on PostgreSQL as its primary database, interfaced via Prisma ORM, providing type-safe and efficient data management.

This platform enables users to offer, request, and exchange skills or services, utilizing a time-credit system alongside direct interactions. The project implements comprehensive features from authentication, session management, and real-time communication to complex entity models including posts, contracts, escrow systems, and chat.

2. Technology Stack

The backend utilizes a modern and highly scalable technology stack:

Core Technologies

- **Runtime:** Node.js
- **Language:** TypeScript
- **Web Framework:** Express.js (v5.x)
- **Database:** PostgreSQL
- **ORM:** Prisma (v7.x) with PostgreSQL adapter

Authentication & Security

- **Authentication:** JSON Web Tokens (JWT) for access tokens and stateful refresh tokens stored in the database.
- **Password Hashing:** bcrypt
- **Validation:** zod for rigorous runtime payload validation.
- **Rate Limiting:** express-rate-limit to protect sensitive endpoints (e.g., login, password reset).
- **CORS:** Cross-Origin Resource Sharing handled via cors.

Real-Time & Utilities

- **Real-Time Communication:** socket.io for bidirectional event-based communication (e.g., chat, notifications).
- **Email Services:** nodemailer and resend for transactional emails like password resets.
- **Task Scheduling:** node-cron for running recurring background jobs.
- **Unique Identifiers:** uuid
- **User Agent Parsing:** ua-parser-js (useful for tracking sessions and device info).

Development & Tooling

- **Execution Engine:** tsx for executing TypeScript files seamlessly.
- **Development Server:** nodemon for auto-reloading during development.
- **Testing:** Native Node.js test runner (tsx --test) and supertest.
- **Environment Management:** dotenv

- **Containerization:** Docker Compose for localized PostgreSQL database provisioning.

3. Database Schema Overview

The database design encapsulates users, skills, interactions (posts), negotiations (chat), and fulfillment (contracts, exchanges).

Core Entities:

- **User:** Stores profile information, available balances, escrow balances, and metrics (services provided/received).
- **Skill / UserSkill:** Represents normalized skills and links them to users as either an OFFER or a REQUEST.
- **Session:** Tracks refresh tokens, device info, and IP addresses to manage active user logins securely.
- **PasswordResetToken:** Manages hashed tokens used for the "forgot password" flow.
- **Post / SavedPost:** Users can create posts categorized as OFFER or REQUEST. Posts contain required time credits and can be saved by other users.
- **Transaction:** Logs the movement of time credits or balances (types include TRANSFER, REFUND, WELCOME_BONUS).
- **Conversation / Message / MessageReadReceipt:** Supports a fully featured chat system between users. Conversations can be linked to specific posts.
- **ServiceExchange / WorkSession:** Represents the agreed contract between a provider and a consumer. Includes escrow status, time credits, and chronological work sessions.
- **Review:** Enables users to leave ratings and comments after a ServiceExchange is completed.
- **Notification:** System alerts for users regarding messages, exchange requests, sessions, and contracts.

4. Modules & Architecture

The application is structured into domain-specific modules located in `src/modules`. Each module is self-contained, managing its own routing, controllers, and business logic.

4.1 Auth Module (`src/modules/auth`)

Handles user identity and session management.

- **Features:** Registration, login, JWT issuance, refresh token rotation, secure logout, password reset flow via email.
- **Key Concepts:** Distinguishes between short-lived access tokens and long-lived refresh tokens securely stored as HTTP-only cookies and in the database.

4.2 Users Module (`src/modules/users`)

Manages user profiles and public data.

- **Features:** Fetching authenticated user payload (`/me`), retrieving public profiles, updating bio/location, and managing user-specific metrics.

4.3 Skills Module (`src/modules/skills`)

Handles the dictionary of available skills and links them to users.

- **Features:** Creating standardized skills (general or technical), and allowing users to attach skills to their profiles as offerings or requests.

4.4 Posts Module (src/modules/posts)

The central marketplace module.

- **Features:** Creation, querying, and updating of service posts. Supports online/offline service modes, assigning time credits, and saving posts.

4.5 Chat Module (src/modules/chat)

Facilitates direct messaging between users.

- **Features:** Establishing conversations (direct or post-linked), sending messages, marking messages as read/delivered.
- **Integration:** Deeply integrated with `socket.io` to provide real-time updates to participants.

4.6 Contracts & Exchanges Modules (src/modules/contracts & src/modules/exchanges)

The backbone of the service fulfillment process.

- **Features:** Transitioning a post agreement into an active `ServiceExchange`. Handles status transitions (Pending -> Accepted -> In Progress -> Completed).
- **Sub-features:** Work sessions tracking for recording hours spent on a contract, and an escrow system for holding and releasing time credits securely.

4.7 Wallet & Transactions Module (src/modules/wallet)

Manages the internal economy of the platform.

- **Features:** Processing transactions between users, issuing welcome bonuses, and executing refunds. Deducts or adds to `available_balance` and `escrow_balance`.

4.8 Notifications Module (src/modules/notifications)

Alerts users to important platform events.

- **Features:** Generating notifications for new messages, exchange requests, deadline proposals, and system alerts. Integrates with real-time sockets for instant delivery.

4.9 Reviews Module (src/modules/reviews)

Handles post-exchange feedback.

- **Features:** Allowing consumers and providers to rate and comment on completed service exchanges, helping build community trust.

4.10 Recommender Module (src/modules/recommender)

Likely utilizes algorithms or data matching to suggest posts, skills, or users to other users to increase platform engagement.

5. Security & Infrastructure

Security Measures

- **Hashing:** Passwords are never stored in plain text (bcrypt). Reset tokens are also hashed in the database before storage.
- **Session Management:** Refresh tokens can be revoked, ensuring compromised sessions can be terminated.
- **Input Validation:** Every incoming request payload is strictly validated against zod schemas to prevent injection attacks and ensure data integrity.
- **Rate Limiting:** Applied to authentication routes to mitigate brute-force attacks.

Real-Time Infrastructure

The real-time aspect (`src/realtime`) is powered by Socket.io. It listens for events such as `connection`, `disconnect`, and custom chat/notification events to push updates directly to the client without polling.

Scripts and Tooling

The `scripts` directory contains utilities for setting up the environment:

- Database seeding (`seed-arabic-demo.ts`, `seedWaslaData.ts`).
 - Production deployment scripts (`migrate-deploy.sh`).
 - User cleanup scripts.
-

6. Conclusion

Wasla Backend is architecturally mature, utilizing industry standards for web applications. The clear separation of concerns (Auth, Chat, Wallet, etc.) alongside a robust database schema (Prisma) provides a solid foundation for building a highly interactive, secure, and scalable service exchange platform.